Tema 02: Funciones

METODOLOGÍA DE LA PROGRAMACIÓN

GRADO EN INGENIERÍA INFORMÁTICA

DECSAI

UNIVERSITAS GRANATENSIS · 1531 · PLUS ULTRA

Índice

- Nociones básicas.
- La función main.
- La pila.
- Paso de parámetros y devolución de resultados.
- Funciones inline.
- Parámetros con valor por defecto.
- Punteros a función.

Nociones básicas

Declaración vs definición

- **Declaración**: Le indicamos al sistema la existencia de una función con un determinado número de parámetros y un determinado parámetro de retorno.
  - No es necesario indicar el nombre de las variables.
  - Una vez declarada una función puede utilizarse sin problema dentro del código.

- **Definición**: Definimos el código de la función.
  - Los parámetros deben tener nombre ya que se usarán dentro del código de la función.
  - Si una función está declarada, su definición puede realizarse después de ser llamada.

Nociones básicas

Declaración vs definición

```
using namespace std;

float suma (float, float);
float resta(float a, float b); //no hace falta poner nombres a las variables
                               //pero se pueden poner

int main(){
    suma(5,4);
    resta(8,3);
}

float suma(float a, float b){
    return a+b;
}
float resta(float a,float b){
    return a-b;
}
```

Nociones básicas

Sobrecarga de funciones

- Puede haber funciones diferentes con el mismo nombre siempre y cuando tengan distintos parámetros de llamada.

- Ejemplo:

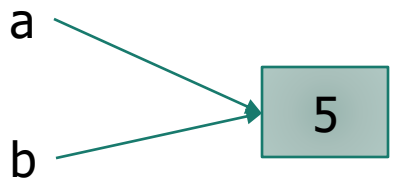
```
int suma(int a, int b){
        return a+b;
}
int suma(int a, int b, int c){
    return a+b+c;
}
float suma(float a, float b){
    return a+b;
}
/*double suma(float a, float b){ //error
    return a+b;
}*/
```

Mismos parámetros

Nociones básicas

Tipo de dato referencia

- Es una variable que hace referencia a otra variable.
- Podemos cambiar el valor referenciado usando cualquiera de los dos identificadores.
- No usarlo sin motivo, puede generar muchos errores.
- Se diferencia de los punteros en que los punteros lo que almacenan son direcciones de memoria.
- Deben asignarse a un objeto existente y no puede reasignarse.



Nociones básicas

Tipo de dato referencia

```
int main(){
    int a = 6;
    int &b = a;

    b = 8;
    cout << a << " " << b << endl;
}
```

```
8 8

Process returned 0 (0x0)   execution time : 1.773 s
Press any key to continue.
```

- **Referencia constante**
  - Podemos modificar el valor usando la variable original pero no la referencia:

```
//referencia constante
const int &c = a;
//c = 9 //ERROR
a = 15;
cout << c << " " << a << endl;
```

```
15 15

Process returned 0 (0x0)   execution time : 1.267 s
Press any key to continue.
```

Nociones básicas

DECSAI

Tipo de dato referencia

- Una función puede devolver referencias a variables.
- Es importante que la referencia que se devuelva no pertenezca a una variable que ya no exista (por ejemplo, una variable local de la función).
- Esa referencia puede utilizarse para modificar el valor referenciado.
- El uso de funciones con referencias tiene más sentido en clases, para permitir que se modifiquen los datos miembro (lo veremos en el tema 3).

Nociones básicas

Ejemplo de uso

```
#include <iostream>

using namespace std;

int & modificaValor(int * v, int pos){
    return v[pos];
}

int main(){
    int v[5] = {1,2,3,4,5};
    modificaValor(v,2) = 4;
    for(int i = 0; i < 5;i++){
        cout << v[i] << " ";
    }
    cout << endl;
    int & a = modificaValor(v,1);
    a = 0;
    for(int i = 0; i < 5;i++){
        cout << v[i] << " ";
    }
}
```

```
1 2 4 4 5
1 0 4 4 5
Process returned 0 (0x0)   execution time : 1.581 s
Press any key to continue.
```

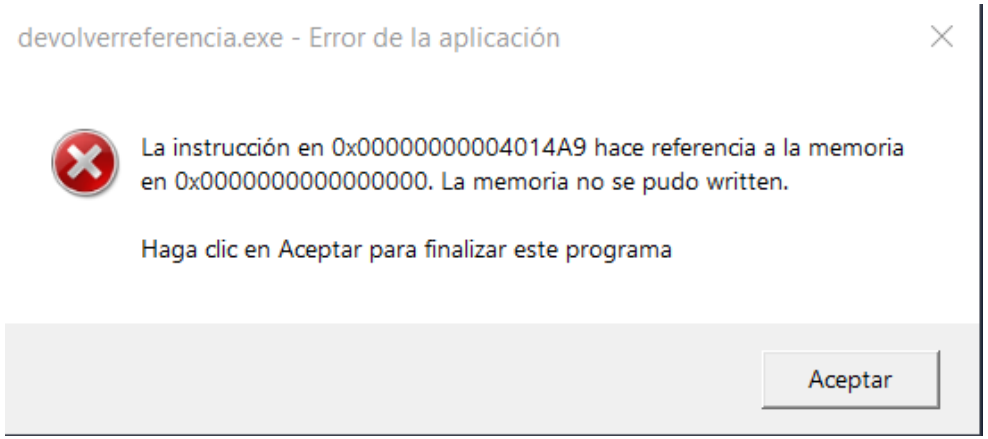
Nociones básicas

Ejemplo de uso erróneo

```
int & prueba(){
    int valor = 0;
    return valor;
}

int main(){

    int & b = prueba();
    b = 4;
```




```
Process returned -1073741819 (0xC0000005)   execution time : 8.516 s
Press any key to continue.
```

- No se debe utilizar una referencia a un valor ya no existente.
- Hacerlo produce error en tiempo de ejecución.

Nociones básicas

Devolución de referencia constante

- En caso de realizarse una devolución de una referencia constante, éstas no puede utilizarse para modificar el valor, sólo para lectura del mismo.

```
const int & observaValor(int *v, int pos){
    return v[pos];
}


int main(){

    int v[5] = {1,2,3,4,5};
    observaValor(v,2) = 4; //Esto da error

    cout << observaValor(v,2) << endl;
```

error: assignment of read-only location 'observaValor(((int*)(& v)), 2)'|

Esto si es correcto

La función main

- Es la función que nos sirve como punto de partida para la ejecución de un programa.
- Debe aparecer en el programa para que la compilación pueda llevarse a cabo.
- La función main:
  - No se puede sobrecargar.
  - No se puede declarar como inline.
  - No se puede declarar como static.
  - No se puede llamar desde el programa.

La función main

DECSAI

Argumentos de la función main

```
int main(int argc, char* argv[])
```

- **argc**
  - Número de argumentos expuestos en argv
- **argv**
  - Vector de argumentos introducidos.
  - Cada argumento es un char *.
- La función main devuelve 0 si toda la ejecución ha ido bien o un código de error en caso contrario.

La función main

- Los argumentos de main se indican como una lista de valores después de llamar al ejecutable del programa.

```
> miPrograma <argumento1> <argumento2> … <argumentoN>
```

- Algunos ejemplos de llamada:
  - > sumaValores 6 7 8 9 10 7 5
  - > buscaPersona 76467743C
  - > buscaPersonaPorApellidos Morente Sánchez

La función main

Ejemplo de muestra de datos por pantalla con conversión a string

```
int main(int argc, char * argv[])
{
  //declaramos un vector para almacenar los argumentos
    string * setdeargumentos = new string[argc];
    //pasamos todos los argumentos a string
    for(int i = 0; i < argc; i++){
        setdeargumentos[i] = string(argv[i]);
    }
    //imprimimos valores
    for(int i = 0; i < argc;i++){
        cout << "argumento " << i << " " << setdeargumentos[i] <<  endl;
    }
}
```

```
argumento 0 E:\pruebacodeblock\pruebaargumentos\bin\Debug\pruebaargumentos.exe
argumento 1 argumento1
argumento 2 45
argumento 3 valor

Process returned 0 (0x0)   execution time : 1.846 s
Press any key to continue.
```

La función main

Conversión de valores tipo cadena o string a datos numéricos y viceversa

- **De char * a int**: atoi.
- **De string a int**: stoi.
- **De string a float**: stof.
- **De char * a float**: atof.
- **De string a double**: stod.
- **De string a long double**: stold.

```
int main(){
    const char * cadena = "esto es una cadena";
    const char * cadenanum = "344";

    string cadenastring = string(cadena);
    string numerostring = string(cadenanum);

    int valornumerico = atoi(cadenanum);
    int valornumerico2 = stoi(numerostring);

    float valorflotante = atof(cadenanum);
    float valorflotante2 = stof(numerostring);
}
```

La función main

DECSAI

Conversión de datos numéricos a string

- Para convertir cualquier tipo numérico a string utilizamos la función to_string().

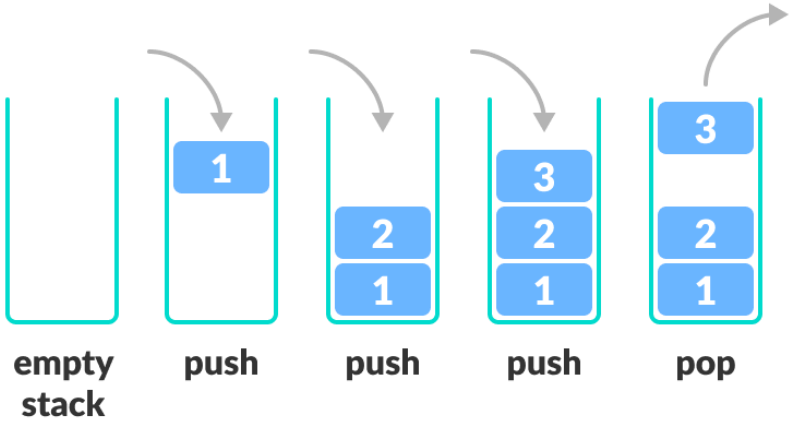
```
string cadenaapartirdenumero = to_string(valorflotante);
string cadenaapartirdeentero = to_string(valornumerico);
```

- Gracias a las funciones de conversión vistas podemos obtener toda la información de argv y almacenarla en las variables apropiadas.

- El primer argumento, argv[0], siempre contiene la ruta del ejecutable por lo que todos los argumentos introducidos aparecerán de argv[1] en adelante.

La Pila

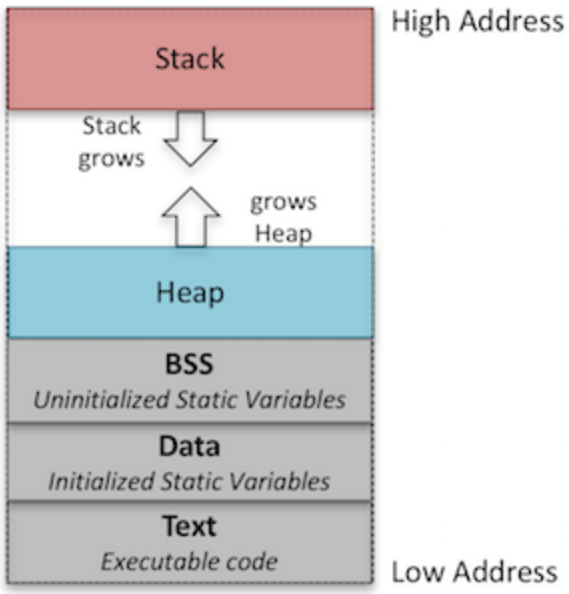
- La pila o stack forma parte de la memoria reservada para la ejecución del programa.
- Se llama pila porque tiene estructura de cola LIFO (last in, first out).



https://stackoverflow.com/questions/62499926/confusions-about-stack-implements

La Pila

- Parte de la memoria está destinada a almacenar aquellas variables de ámbito local declaradas dentro de las funciones.
- Cada vez que se llama a una función, se añade su entorno con lo que el tamaño del stack aumenta.
- Cuando una función termina de ejecutarse, se elimina su entorno.
- El manejo de esta memoria la realiza el sistema operativo.

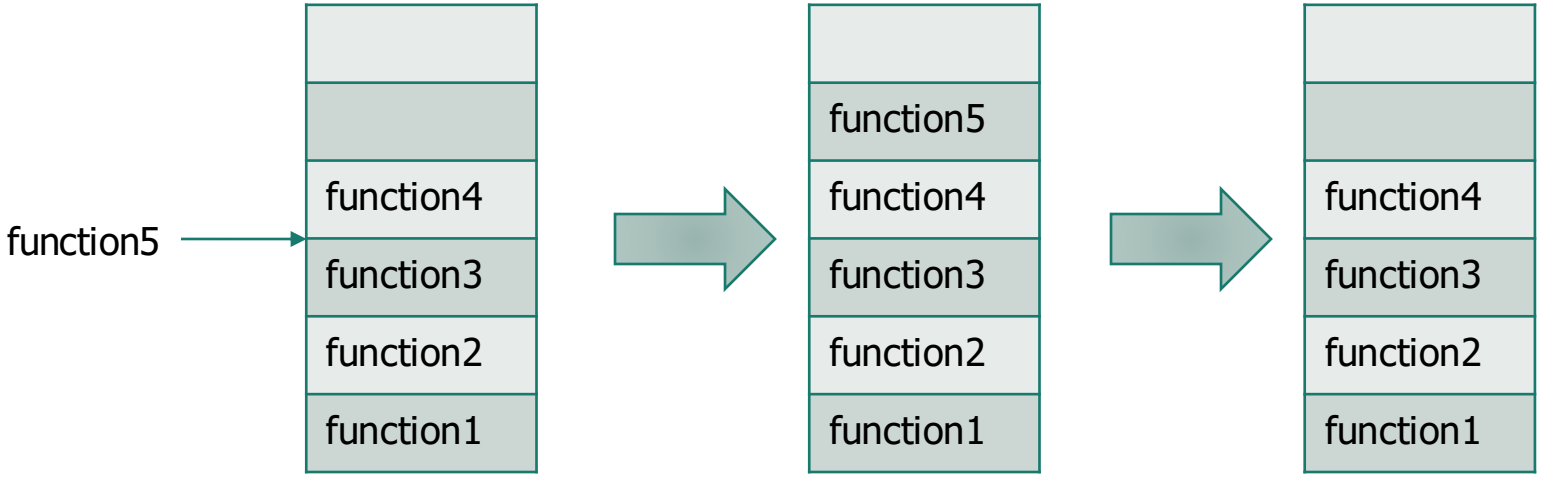


https://study.com/academy/lesson/dynamic-memory-allocation-definition-example.html

La Pila

- Si se ha llamado a 5 funciones, la stack almacena información sobre todos sus entornos siendo, la última función llamada, la que estaría localizada en lo alto de la pila.

- Aunque la pila tiene mucho espacio, este no es infinito por lo que se puede desbordar.



La Pila

DECSAI

Partes del entorno de una función dentro de la pila

- **Variables locales.**
  - Variables declaradas dentro de la función.
- **Dirección de retorno.**
  - Dirección donde está localizado el código que debe ejecutarse una vez terminada la ejecución de la función.
  - Generalmente es la dirección del trozo de código de la función que llamó a la función que se está ejecutando en este momento.
- **Parámetros**
  - Los parámetros de la función toman los valores enviados y pasan a formar parte de las variables accesibles por el código de la función.

La Pila

DECSAI

Puntero a frame y puntero a stack

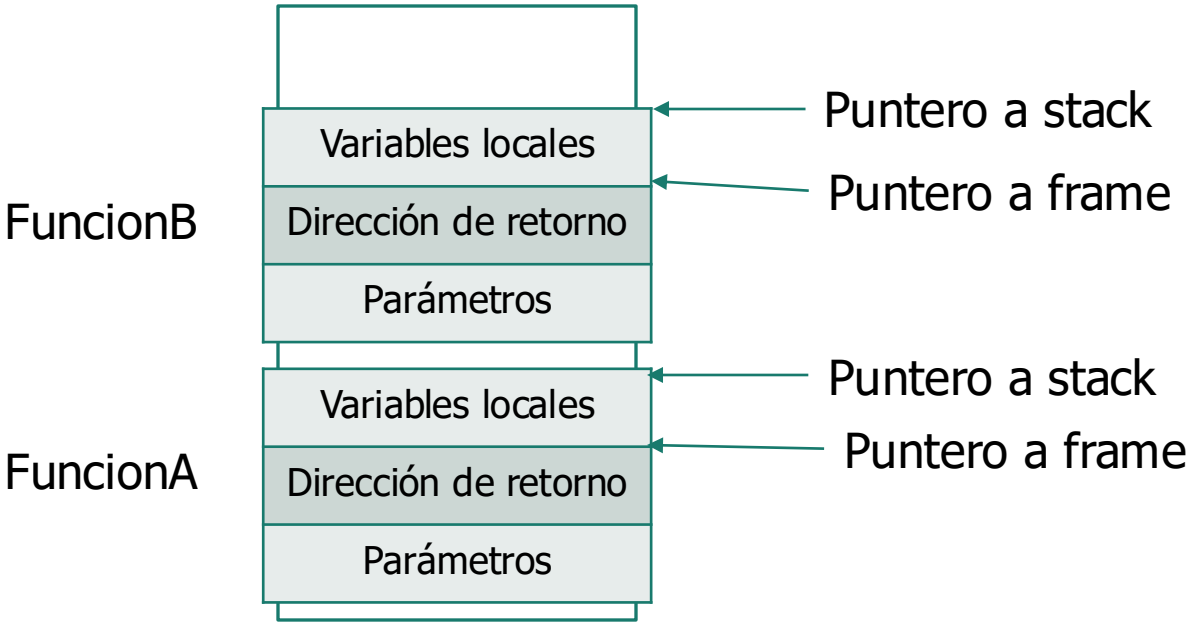
- Para poder manejar adecuadamente la pila, el sistema utiliza dos punteros distintos: el puntero a frame y puntero a stack.

- **Puntero a frame (frame pointer)**
  - Puntero que indica la dirección en donde está almacenada la función dentro del stack.
  - Fijo durante toda la ejecución de la función.
  - Sirve para ir cambiando de una función a otra conforme sea necesario.

- **Puntero a stack (stack pointer)**
  - Puntero que apunta siempre a lo alto de la pila de la función.
  - Va variando en función de los datos que va generando la función.

La Pila

Representación gráfica de la pila tras la ejecución de dos funciones.




```
FuncionA(){
    FuncionB();
}

int main(){
    FuncionA();

}
```

Paso de parámetros a funciones

DECSAI

Resumen de los visto hasta ahora

- Cuando pasamos una variable a una función, si es de tipo simple lo llamamos paso por valor porque se copia el valor a la función manteniéndose el valor original fuera de la función.

- Cuando pasamos un vector a una función, lo que pasamos es su puntero por lo que todos los cambios realizados al vector se mantienen fijos después de que la función se haya ejecutado (ver 10vectoresafunciones.cpp).

- Desventaja: Las funciones (con lo visto hasta ahora) sólo pueden devolver un valor o modificar vectores.

Paso de parámetros a funciones

DECSAI

Paso de valores a funciones por referencia

- Mediante el uso del operador & en la definición de la cabecera de la función, podemos pasar variables no vectoriales por referencia.
- Si una variable que pasamos por referencia se modifica, entonces queda modificada fuera de la función.
- No se debe abusar del paso por referencia ya que rompe la modularidad y hace difícil encontrar fallos en el código.
- Gracias al paso de valores por referencia, una función puede devolver más de un valor.

Paso de parámetros a funciones

Paso de valores por referencia (ejemplo)

```
#include <iostream>

using namespace std;

int division (int dividendo, int divisor, int & resto){
    int resultado = dividendo/divisor;
    resto = dividendo%divisor;
    return resultado;
}

int main(){
    int resto;
    int cociente = division(6,5,resto);
    cout << cociente << " " << resto << endl;
}
```

```
1 1

Process returned 0 (0x0)   execution time : 1.342 s
Press any key to continue.
```

Paso de parámetros a funciones

DECSAI

Paso constante de valores por referencia

- El paso por valor de objetos hace que, cada vez que se llame a una función, se realice una copia física de éste.
- Podemos usar el paso por referencia para mejorar la eficiencia.
- Podemos usar const en el paso por referencia para evitar que se pueda modificar el objeto.
- Gracias a los pasos constante de valores por referencia, es posible prevenir modificaciones indeseadas del objeto.

Paso de parámetros a funciones

Paso constante de valores por referencia

- Ejemplo:

```
#include <string>
#include <iostream>
using namespace std;
struct Persona{
    string DNI;
    string nombre;
};

void imprimePersona(const Persona & a){
    //a.DNI = "3453453"; ERROR
    cout << a.DNI << " " << a.nombre << endl;
}

int main(){
    Persona Pedro = {"56767878G","Pedro"};
    imprimePersona(Pedro);
}
```

Paso de parámetros a funciones

DECSAI

¿Cuándo pasar por referencia?

- El **paso por valor** es recomendable con:
  - Tipos simples cuando no se quiere que el valor se modifique fuera de la función.
- El **paso por referencia** es recomendable con:
  - Tipos simples cuando se quiere modificar su valor fuera de la función.
- El **paso por referencia constante** es recomendable con:
  - Vectores, matrices, objetos … cuando no van a modificarse en la función.

Paso de parámetros a funciones

DECSAI

Paso de objetos como parámetro

- Los objetos son tipos complejos que pueden contener vectores y matrices.
- Cuando pasamos un objeto por valor, se hace una copia de todo su contenido.
- La copia de toda la información puede ser muy ineficiente.
- Es recomendable pasar los objetos por referencia.
- Si no queremos que el objeto se modifique: usamos referencia constante.

Paso de parámetros a funciones

Paso de objetos como parámetro

Ejemplos:

```
void ImprimeHijos(Persona a){ //Ineficiente
    a.Hijos[1].setNombre("Luis"); //el valor permanece cambiado fuera de la
                                  //función porque usamos memoria dinámica
    a.imprimeHijos();
}

void ImprimeIdentidad(Persona & a){ //se pasa la referencia
    cout << a.verDNI() << " " << a.verNombre() << endl;
}

void ImprimeIdentidad (const Persona & a){
//Esta funcion solo puede llamar a métodos const que tenga Persona
}
```

Paso de parámetros a funciones

Paso de objetos con array como parámetro

Si aquí añado & entonces el valor queda cambiado fuera de la función

```
struct Array{
    int v[50] = {1,2,3,4,5};
    int tam = 5;
};

void cambiaArray(Array a){
    a.v[2] = 15;
    for(int i = 0; i < a.tam;i++){
        cout << a.v[i] << " ";
    }
    cout << endl;
}

int main(){
    Array a;
    cambiaArray(a);
    for(int i = 0; i < a.tam;i++){
        cout << a.v[i] << " ";
    }
    cout << endl;
}
```

```
1 2 15 4 5
1 2 3 4 5

Process returned 0 (0x0)   execution time : 1.431 s
Press any key to continue.
```

Funciones inline

DECSAI

Utilidad

- Cada llamada a función implica un nuevo elemento que se agrega al stack.
- Para funciones muy pequeñas, el tener que ser agregadas al stack puede llegar a ser ineficiente.
- Las funciones inline generan una copia del código en el sitio en que son llamadas reduciendo así, el número de llamadas a funciones en el stack.
- Sólo tienen sentido para funciones pequeñas, si las funciones son grandes entonces el coste de replicar el código es mayor que el de realizar la llamada.
- El compilador de C++ ignora el comando inline si se aplica a funciones muy grandes.

Funciones inline

DECSAI

Utilidad

- Utilizamos, por tanto, inline, con funciones pequeñas de uso frecuente.
- Si a la función se la llama poco, no tiene sentido evitar que se añada al stack.
- Cómo se define una función inline:

```
inline <tipodevuelto> <nombreFuncion> (<parámetros>)
```

- Simplemente colocamos el comando inline delante de la definición de la función.

Funciones inline

Ejemplo de función inline

```
#include <iostream>

using namespace std;

inline double cubo (const double lado){
   return lado*lado*lado;
}

int main(){
   cout << "Introduzca la longitud del lado del cubo" << endl;
   double valorlado;
   cin >> valorlado;

   cout << "El volumen del cubo es " << cubo(valorlado) << endl;
}
```

Parámetros con valor por defecto

DECSAI

Motivación

- Es frecuente que una función se llame de manera frecuente con algún valor de argumento en particular.
- En caso de que esto ocurra, puede definirse la función con parámetros con valor por defecto.
- Si un usuario no especifica ningún valor para un parámetro por defecto, dicha función se ejecutará con el valor por defecto designado.
- Cuando un programa omite un parámetro que tiene un argumento por defecto, el compilador le asigna dicho argumento al parámetro.

Parámetros con valor por defecto

DECSAI

Especificación de los valores por defecto

- Deben situarse siempre a la derecha del todo de la cabecera de la función.
- Cuando se llama a una función con dos o más argumentos predeterminado, si un argumento omitido no es el argumento que se encuentra más a la derecha, se deben omitir todos los argumentos a la derecha de dicho argumento.
- Los valores predeterminados pueden ser:
  - Constantes.
  - Variables globales.
  - Llamadas a funciones.

Parámetros con valor por defecto

DECSAI

Definición de funciones con parámetros por defecto

<tipo> <identificador>(<parámetros sin argumentos>, <tipo> <nombrevariable> = <valor por defecto>)

- Ejemplos:
  - int division (int dividendo, int & resto, int divisor=2);
  - float areadelcirculo(int radio =1);
  - float volumenCaja(int longitud =1, int ancho=1, int altura=1);
  - String datosPersona(string nombre, string DNI="00000000A");

Parámetros con valor por defecto

Ejemplo de uso

```
int volumencaja(int longitud = 1, int ancho =1, int altura = 1){
    return longitud*ancho*altura;
}

int main(){
    cout << volumencaja() << endl;
    cout << volumencaja(3,4) << endl;
    cout << volumencaja(4,5,3) << endl;
}
```

```
1
12
60

Process returned 0 (0x0)   execution time : 1.490 s
Press any key to continue.
```

Punteros a función

DECSAI

Introducción

- Tanto variables como funciones se almacenan en memoria.
- De igual manera que vimos que podíamos referenciar a la dirección de una variable con un puntero, podemos hacer referencia a la dirección de memoria donde se localiza una función.
- Un puntero a función almacena la dirección en la que comienza el código de una función.
- Los apuntadores a funciones permiten pasar funciones como parámetro.

Punteros a función

DECSAI

Definición de un puntero a función en la cabecera

- Para indicar que una función recibe un puntero a función, se utiliza la siguiente definición de parámetro:

```
<tipo> (*<nombrepuntero>) (<tipoparam1>,<tipoparam2>, …, <tipoparamN>)
```

- En la llamada a la función se introduce como parámetro el nombre de la función a utilizar.

- Ejemplo:
  - void burbuja (int * matriz, const int tamanio, bool (*compara) (int,int))
  - Ejemplo de llamada a función: burbuja(m,15,ascendente);

Punteros a función

DECSAI

Características de los punteros a función

- La función que se le pasa como argumento debe tener el mismo número de parámetros, del mismo tipo y devolver un dato del tipo indicado.
- Siempre que la cabecera sea igual, no hay restricciones en cuanto al código de la función.
- Gracias a los punteros a función, podemos aplicar una función u otra en un punto de un código según algún tipo de condición.
- Una vez pasada la función como argumento, utilizamos el puntero a función para operar.

Punteros a función

Ejemplo simple de puntero a función

```
int suma(int a, int b){
    return a+b;
}
int resta(int a,int b){
    return a-b;
}

int aplicafuncionbinaria(int a, int b, int (*op)(int,int)){
    return op(a,b);
}

int main(){
   cout << aplicafuncionbinaria(5,8,suma) << endl;
   cout << aplicafuncionbinaria(9,8,resta) << endl;
}
```

```
13
1

Process returned 0 (0x0)   execution time : 1.687 s
Press any key to continue.
```

Punteros a función

DECSAI

Vectores de punteros a función

- Al igual que hemos trabajado con vectores de variables, es posible, gracias a los punteros, trabajar con vectores de funciones.

- Puede ser útil a la hora de establecer menús en los que el usuario escoge entre varias opciones.
  - Guardamos lo que hay que hacer para cada opción en el vector y pedimos al usuario que escoja la función que desea.

- Restricción: Todas las funciones deben tener el mismo número de argumentos, mismo tipo en los parámetros y mismo tipo de retorno.

Punteros a función

DECSAI

Vectores de punteros a función

- Podemos definir el vector de funciones como un vector estático inicializado con el operador corchete:

```
<tipo> (*<idvector>[<tamanio>](<param1>,… <paramN>) = {<funcion1>,…,<funcionM>}
```

- Ejemplos:
  - void(*f[3])(int) = {funcion1,funcion2,funcion3};
  - Float(*vf[4])(float,float) = {suma,resta,multiplicacion,division};

Punteros a función

Ejemplo de vectores de punteros a función

```
int suma(int a, int b){
    return a+b;
}
int resta(int a,int b){
    return a-b;
}
int multiplicacion(int a, int b){
    return a*b;
}

int main(){
    int (*operacion[3])(int,int) = {suma,resta,multiplicacion};
    for(int i = 0; i < 3; i++){
        cout << (operacion[i])(6,7) << endl;
    }
}
```

```
13
-1
42

Process returned 0 (0x0)   execution time : 1.342 s
Press any key to continue.
```